

CSE 25 - Introduction to Artificial Intelligence

Week 7 Thursday: Evaluating language models and exploring semantic models

Learning Objectives:

- Compute and interpret log-probability and perplexity for language models
- Describe a strategy for incorporating Out-of-Vocabulary tokens in language models
- Explain the concept of a semantic space for words
- Visualize and interpret simple word embeddings
- Measure similarity between words using distance and dot product
- Identify and discuss bias in word embeddings and its implications

Instructions

Use your copy of this notebook on Datahub and complete it during class. Work through the cells below **in order**. You may discuss with your neighbors, but make sure you understand each step yourself.

SUBMISSION: When finished, download the notebook to have a local copy for your records. Choose two cells where you wrote code or answers and take screenshots of them to upload to Gradescope under **In-Class – Week 7 Thursday** to receive credit.

Evaluating Language Models

There are two main ways to evaluate a language model.

1. Extrinsic Evaluation

The most meaningful way to evaluate a language model is to embed it inside a real application and measure task performance.

Examples:

- Use it in speech recognition → measure transcription accuracy.
- Use it in machine translation → measure translation quality.

If one language model leads to better downstream performance, it is the better model.

This is called **extrinsic evaluation**.

- Measures real-world impact

- Often expensive and slow

2. Intrinsic Evaluation

Instead of evaluating the full system, we evaluate the language model by itself.

The main idea:

A better language model assigns higher probability to real, unseen text.

Given a test corpus $W = w_1, w_2, \dots, w_N$, we compute: $P(W)$

The model that assigns **higher probability** to the test corpus is better. This is called **intrinsic evaluation**.

However, raw probabilities of long sequences become extremely small and hard to interpret. So instead of raw probability, we use a normalized metric called *Perplexity*. Perplexity is the standard intrinsic evaluation metric for language models.

Reminder: Train, Validation, and Test Splits

As we have seen before, we use three distinct datasets:

Training Set

- Used to learn model parameters
- For n-grams: used to compute counts and probabilities

Validation Set

- Used to tune hyperparameters
- Used to compare model variants
- Helps prevent overfitting

Test Set

- Completely held out
- Used only once at the very end
- Provides an unbiased estimate of generalization

We must **never** train on the test set. Doing so causes *data leakage*, which artificially inflates probabilities and leads to misleading evaluation results.

Note on Out of Vocabulary (OOV) Tokens

Some tokens/words may appear in validation or test data that were not seen during training. If a word has zero count, the model assigns it probability 0, which makes the entire sequence probability 0.

To handle this, we introduce a special token `<UNK>` :

- During training, rare words are replaced with `<UNK>` .
- The model learns a probability for `<UNK>` .
- Any unseen word in validation or test data is mapped to `<UNK>` .

Perplexity (PPL)

When evaluating a language model, we want to know how **surprised** the model is by a corpus. If the model assigns high probability, it is not very surprised. If it assigns low probability, it is very surprised.

A better language model is therefore one that is **less surprised** by real, unseen text.

Surprise is inversely related to probability: $\frac{1}{P(W)}$

However, value of $P(W)$ depends on sequence length of W .

Longer texts automatically have smaller probabilities. To remove the effect of length, we normalize (take the geometric mean) per token by taking the N^{th} root.

This gives the definition of **perplexity**:

$$\begin{aligned} \text{PPL}(W) &= P(W)^{-\frac{1}{N}} \\ &= \sqrt[N]{\frac{1}{P(W)}} \end{aligned}$$

Lower perplexity indicates that the model assigns higher probability to the corpus and is therefore a better predictor of the data.

Log Form of Perplexity (Used in Practice)

In practice, we use the log form. Using the chain rule,

$$P(W) = \prod_{t=1}^N P(w_t \mid \text{history})$$

Taking logs,

$$\log P(W) = \sum_{t=1}^N \log P(w_t \mid \text{history})$$

Substituting into the definition,

$$\text{PPL}(W) = \exp\left(-\frac{1}{N} \sum_{t=1}^N \log P(w_t \mid \text{history})\right)$$

```

In [1]: # The toy example from before
tokenized_toy_sentences = [
    ["<s>", "to", "be", "or", "not", "to", "be", "</s>"],
    ["<s>", "to", "be", "a", "king", "</s>"],
    ["<s>", "to", "eat", "pizza", "</s>"],
]

all_tokens = []

# Create a list of all tokens in the corpus
for sentence in tokenized_toy_sentences:
    for token in sentence:
        all_tokens.append(token)

vocab = set(all_tokens)
vocab_order = sorted(vocab)
vocab_size = len(vocab)

# Initialize a dictionary to count the occurrences of each word
word_counts = {}
for w in all_tokens:
    # If it's the first time we see this word, initialize its count to 1
    if w not in word_counts:
        #REMOVE FROM STUDENT VERSION
        word_counts[w] = 1 # YOUR CODE HERE

        #word_counts[w] = None # YOUR CODE HERE
    # else, increment the existing count by 1
    else:
        #REMOVE FROM STUDENT VERSION
        word_counts[w] += 1 # YOUR CODE HERE

        #word_counts[w] = None # YOUR CODE HERE

# Now let's estimate bigram probabilities by counting bigrams and dividing by

bigram_counts = {}

# Initialize bigram counts for all possible bigrams in the vocabulary
for prev_word in vocab:
    for curr_word in vocab:
        # We can use a tuple as the key for bigram counts
        # since tuples are immutable and can be used as dictionary keys
        bigram_counts[(prev_word, curr_word)] = 0 # Initialize count to 0 for

# Count bigrams in the tokenized sentences (our text corpus)
for sent in tokenized_toy_sentences:
    for i in range(len(sent) - 1):
        prev_word = sent[i] # w_{t-1}
        curr_word = sent[i + 1] # w_t
        new_key = (prev_word, curr_word) # (w_{t-1}, w_t)

        # Update bigram counts

```

```

        if (prev_word, curr_word) not in bigram_counts:
            bigram_counts[(prev_word, curr_word)] = 1
        else:
            bigram_counts[(prev_word, curr_word)] += 1

# Now we can compute the bigram probabilities
# by dividing the count of each bigram by the count of the previous word
#  $P(w_t | w_{t-1}) = \text{Count}(w_{t-1}, w_t) / \text{Count}(w_{t-1})$ 

bigram_probs = {}
for (prev_word, curr_word) in bigram_counts:
    denom = word_counts[prev_word]
    bigram_probs[(prev_word, curr_word)] = bigram_counts[(prev_word, curr_word)] / denom

# Incorporate smoothing
# 1. Add 1 to all bigram counts
bigram_counts_smoothed = {}
for (prev_word, curr_word) in bigram_counts:
    bigram_counts_smoothed[(prev_word, curr_word)] = bigram_counts[(prev_word, curr_word)] + 1

# 2. Add |V| to all word counts to account for the added counts in bigrams
word_counts_smoothed = {}
for prev in word_counts:
    word_counts_smoothed[prev] = word_counts[prev] + vocab_size # Add 1 for

# 3. Recompute bigram probabilities with smoothing
bigram_probs_smoothed = {}
for (prev_word, curr_word) in bigram_counts_smoothed:
    bigram_probs_smoothed[(prev_word, curr_word)] = bigram_counts_smoothed[(prev_word, curr_word)] / word_counts_smoothed[prev_word]

```

```

In [2]: import math

def perplexity(tokenized_sequence, bigram_probs):
    """
    Compute bigram perplexity for a tokenized sequence.
    """
    N = len(tokenized_sequence) - 1 # number of bigram transitions
    log_prob_sum = 0.0
    for i in range(N):
        p = bigram_probs.get((tokenized_sequence[i], tokenized_sequence[i+1]), 1.0)
        log_prob_sum += math.log(p)
    return math.exp(-log_prob_sum / N)

# Compare two sequences
seq_in_train = ["<s>", "to", "be", "or", "not", "to", "be", "</s>"]
seq_out_train = ["<s>", "to", "eat", "a", "pizza", "</s>"]

print("PPL (in-training corpus):", perplexity(seq_in_train, bigram_probs_smoothed))
print("PPL (unseen bigrams):", perplexity(seq_out_train, bigram_probs_smoothed))

PPL (in-training corpus): 4.702917530844674
PPL (unseen bigrams): 6.855290957272971

```

Q. Does the sequence from the training corpus or the unseen sequence have lower perplexity. Why?

Limitations of N-gram Models

- The number of parameters grows rapidly with vocabulary size
- Data sparsity becomes severe for large n
- Long-range dependencies are ignored
- Words are treated as discrete symbols
- No notion of similarity between words

These last two limitations motivate a different approach to language models: instead of counting words we *represent* them with richer data structures.

How can AI systems address these limitations

Remember Semantris (<https://research.google.com/semantris/>, where the system seemed to know which words were related).

How could a computer measure the **similarity between words**?

One idea is to represent words as **points in a space**, where similar words are closer together.

We first explore a **simplified semantic space** where each word is represented by coordinates.

Building a Semantic Space

Consider the words:

man, woman, boy, girl

What features might distinguish these words?

Discuss with your group and write two possible dimensions.

- Dimension 1: YOUR ANSWER HERE
- Dimension 2: YOUR ANSWER HERE

Word	(x)	(y)	Coordinates
man	1	7	[1, 7]
woman	9	7	[9, 7]
boy	1	2	[1, 2]
girl	9	2	[9, 2]

We can plot these on a graph where the x-axis represents **gender** and the y-axis represents **age**.

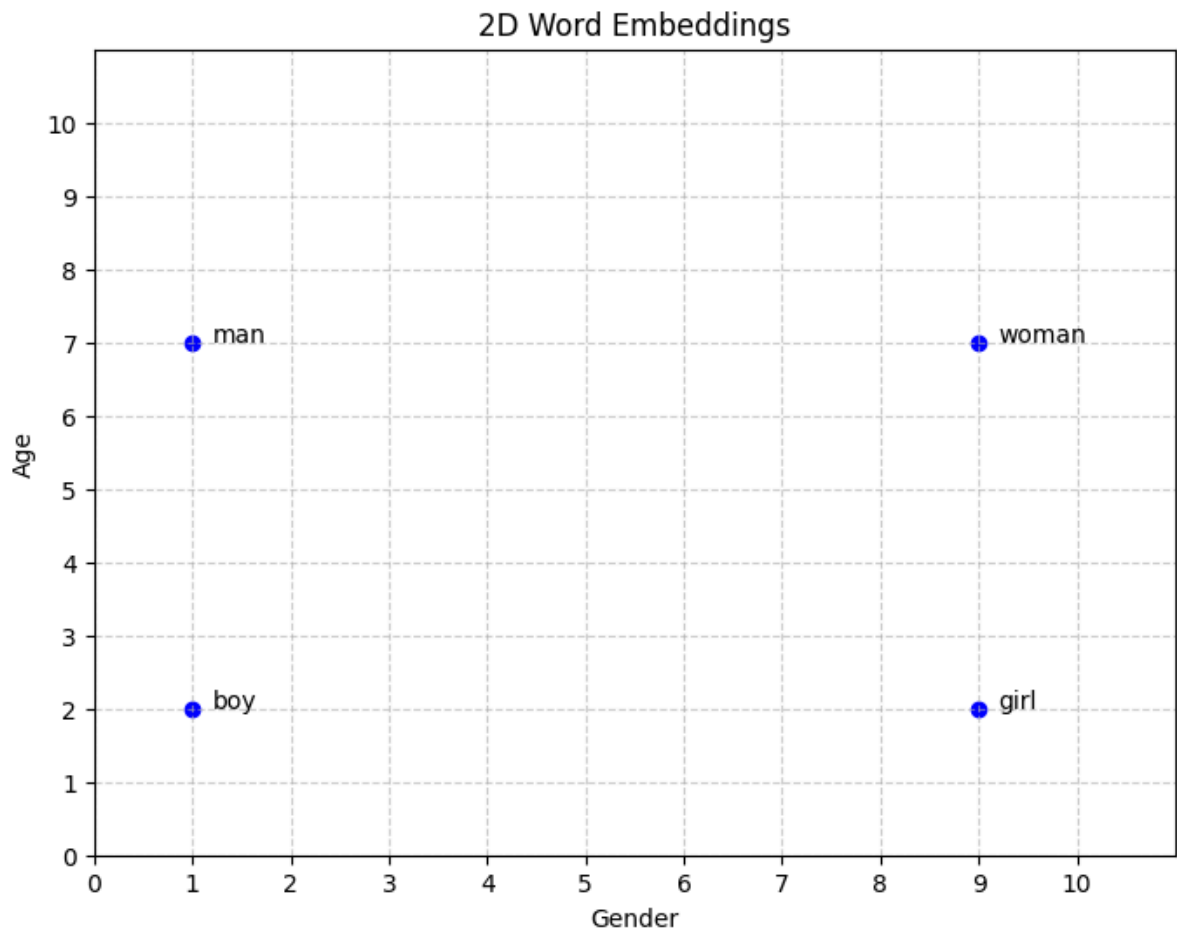
```
In [3]: import matplotlib.pyplot as plt
import math

words_2d = {
    "man": [1, 7],
    "woman": [9, 7],
    "boy": [1, 2],
    "girl": [9, 2]}

def plot_space(data, title):
    plt.figure(figsize=(8, 6))
    for word, coords in data.items():
        plt.scatter(coords[0], coords[1], color='blue')
        plt.text(coords[0] + 0.2, coords[1], word, fontsize=10)

    plt.xlim(0, 11); plt.ylim(0, 11)
    plt.xlabel("Gender")
    plt.ylabel("Age")
    plt.xticks(range(0, 11, 1))
    plt.yticks(range(0, 11, 1))
    plt.title(title)
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.show()

plot_space(words_2d, "2D Word Embeddings")
```



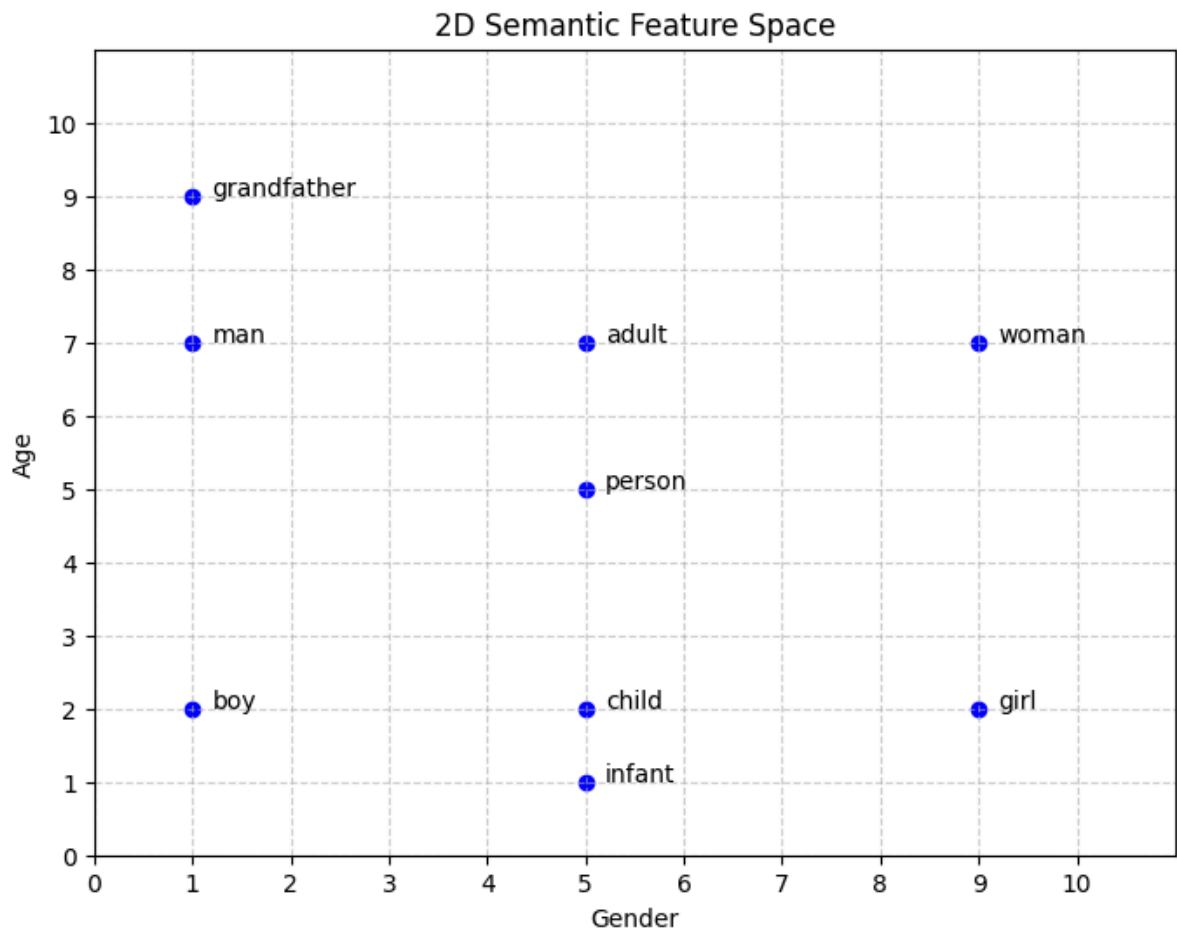
In [4]: *# Let's add more words to the space, and see how they relate to the original*

Table of Word Coordinates from the tutorial

```
additional_words = {  
    "person": [5, 5],  
    "grandfather": [1, 9],  
    "adult": [5, 7],  
    "child": [5, 2],  
    "infant": [5, 1]  
}
```

Add the additional words to the original dictionary
words_2d.update(additional_words)

Plot the updated space with the new words
plot_space(words_2d, "2D Semantic Feature Space")



Q. How would you represent the words "grandmother", "grandparent", "teenager", and "octogenarian"?

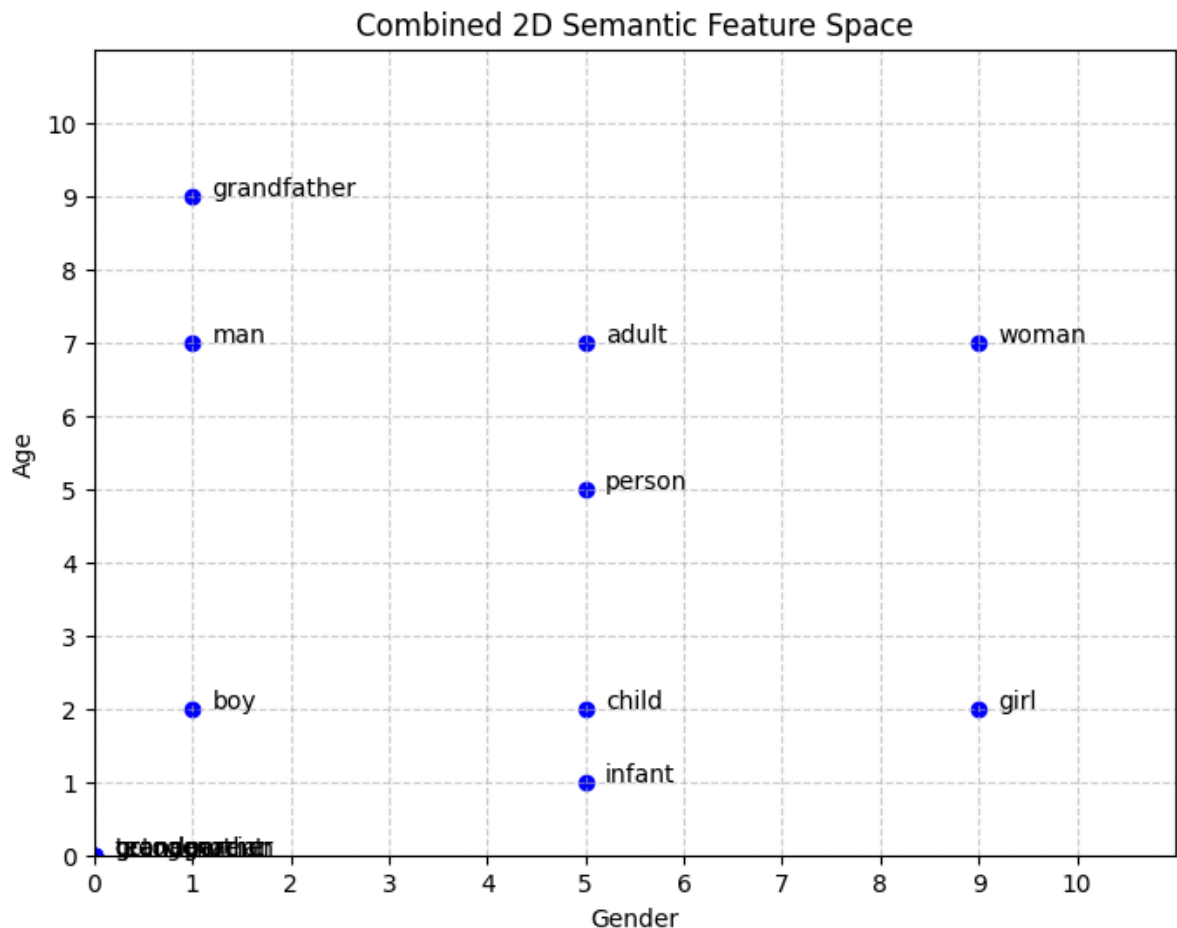
YOUR ANSWER HERE

Hint: Use the same coordinate system as the table above, where $x \approx 1$ for masculine, $x \approx 9$ for feminine, $x \approx 5$ for gender-neutral. $y \approx 1-2$ for very young, $y \approx 7$ for adult, $y \approx 9-10$ for elderly.

```
In [5]: # Change these coordinates to reflect the semantic features of the new words
# YOUR CODE HERE
additional_words = {
    "grandmother": [0, 0],
    "grandparent": [0, 0],
    "teenager": [0, 0],
    "octogenarian": [0, 0]
}

# Combine with original words for plotting
combined_words = {**words_2d, **additional_words}

plot_space(combined_words, "Combined 2D Semantic Feature Space")
```



Now that each word has coordinates in this space, we can measure how **similar two words are** by measuring the **distance between their points**.

Measuring Distance Between Words

So far we have represented words as **points in a semantic space**. Another way to think about these points is as **vectors**.

A **vector** is an arrow that starts at the origin and ends at a point.

For example, if the word **child** has coordinates [5, 2] we can draw it as an arrow from the origin [0, 0] to the point [5, 2].

In this way, every word becomes a vector pointing to its position in the semantic space.

Below we visualize all the words in our 2D semantic space as vectors.

```
In [6]: # Plot words as points and vectors from origin (0,0)
def plot_words_from_origin(words):
    plt.figure(figsize=(8, 6))

    for word, (x, y) in words.items():
        # Vector arrow from origin to the word point
        plt.quiver(0, 0, x, y, angles='xy', scale_units='xy', scale=1,
```

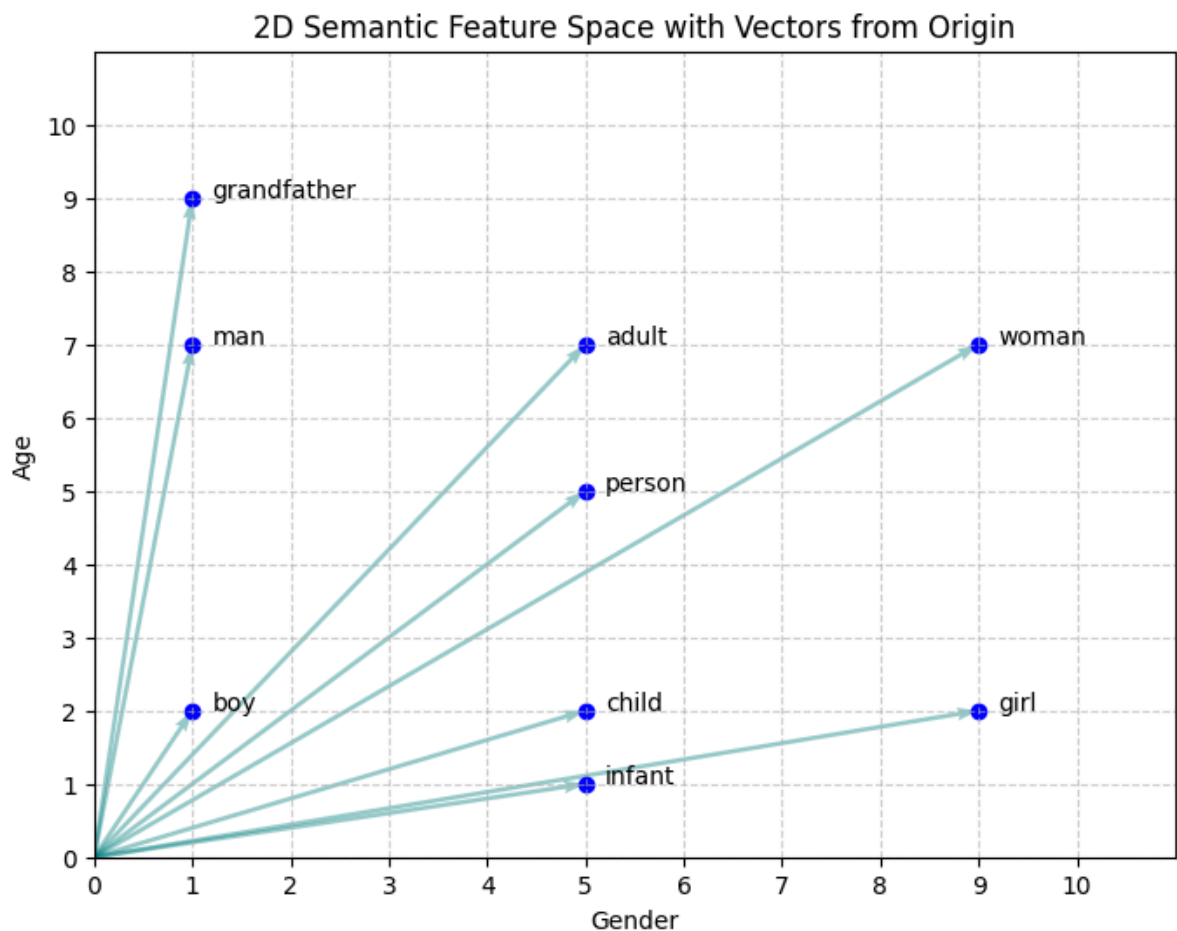
```

        color='teal', alpha=0.4, width=0.004)
    # Point + label
    plt.scatter(x, y, color='blue')
    plt.text(x + 0.2, y, word, fontsize=10)

    plt.xlim(0, 11)
    plt.ylim(0, 11)
    plt.xlabel("Gender")
    plt.ylabel("Age")
    plt.xticks(range(0, 11, 1))
    plt.yticks(range(0, 11, 1))
    plt.title("2D Semantic Feature Space with Vectors from Origin")
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.show()

plot_words_from_origin(words_2d)

```



Q. If two words have **similar meanings**, where should they appear in this space - close together or far apart? Why?

YOUR ANSWER HERE

Measuring Euclidean Distance

We can compare two words by measuring the **distance between their points** in the semantic space.

For example:

- **boy** has coordinates [1, 2]
- **infant** has coordinates [5, 1]

To compute the vector from **boy** → **infant**, subtract the coordinates:

$$[5, 1] - [1, 2] = [4, -1]$$

The length of a vector [x, y] is the Euclidean distance between the points:

$$\sqrt{x^2 + y^2}$$

So the distance between **boy** and **infant** is:

$$\sqrt{4^2 + (-1)^2} = \sqrt{17} \approx 4.12$$

Here are the distances from **boy** to several other words:

Word	Distance from "boy"
child	4.00
infant	4.12
man	5.00
adult	6.40
grandfather	7.00
girl	8.00
woman	9.43
person	5.00

Q. Which word is **most similar to "boy"** according to this distance measure?

YOUR ANSWER HERE

Q. Which word is **least similar**?

YOUR ANSWER HERE

Q. Does this match your intuition about the meanings of these words?

YOUR ANSWER HERE

```
In [7]: # Plot words as points and vectors from "boy"
plt.figure(figsize=(8, 6))

boy_coords = words_2d["boy"]
```

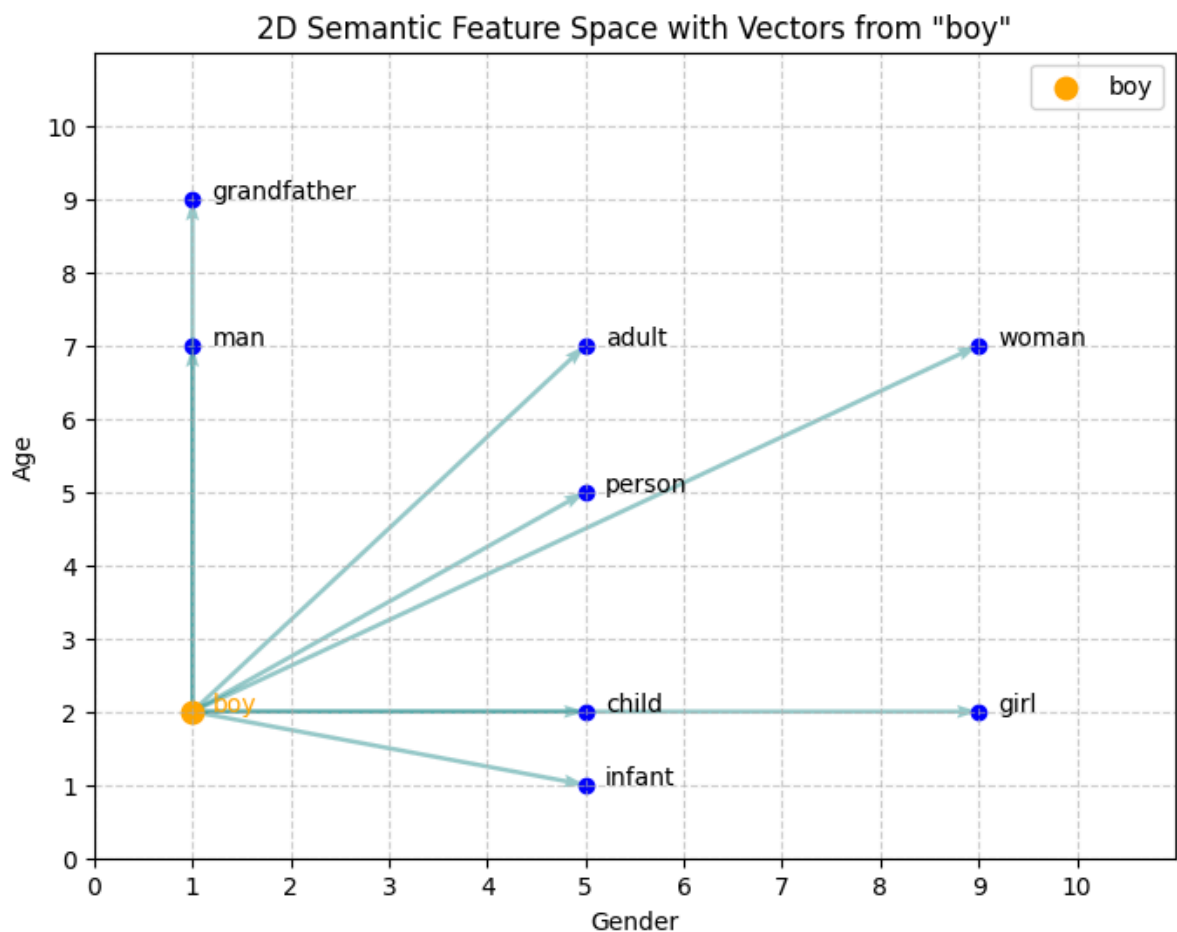
```

for word, (x, y) in words_2d.items():
    if word == "boy":
        continue
    # Vector arrow from "boy" to the word point
    plt.quiver(boy_coords[0], boy_coords[1], x - boy_coords[0], y - boy_coords[1],
              angles='xy', scale_units='xy', scale=1,
              color='teal', alpha=0.4, width=0.004)
    # Point + label
    plt.scatter(x, y, color='blue')
    plt.text(x + 0.2, y, word, fontsize=10)

# Draw "boy" in a different color
plt.scatter(boy_coords[0], boy_coords[1], color='orange', s=80, label='boy')
plt.text(boy_coords[0] + 0.2, boy_coords[1], "boy", fontsize=10, color='orange')

plt.xlim(0, 11)
plt.ylim(0, 11)
plt.xlabel("Gender")
plt.ylabel("Age")
plt.xticks(range(0, 11, 1))
plt.yticks(range(0, 11, 1))
plt.title('2D Semantic Feature Space with Vectors from "boy"')
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.show()

```



Measuring Similarity With the Dot Product

You can see that the closest word to "**boy**" is "**child**", and "**infant**" is only slightly further away.

Euclidean distance is a reasonable way to measure similarity between words.

However, in **real word embeddings**, vectors often differ in **length** as well as direction. Because of this, Euclidean distance is not always the best measure of similarity.

Instead, we often compare vectors using the **dot product**, which measures how well two vectors point in the **same direction**.

If we have two vectors

$$[x_1, y_1] \quad \text{and} \quad [x_2, y_2]$$

their dot product is

$$x_1x_2 + y_1y_2$$

Geometrically, the dot product depends on the **angle between the two vectors**.

- If two vectors point in **similar directions**, the angle between them is small and the dot product is **large**.
- If they point in **very different directions**, the angle is larger and the dot product is **smaller**.

Because of this, the dot product acts as a **similarity measure** rather than a distance measure.

A Problem With Our Current Space

There is one issue with using the dot product in our current semantic space.

All of our word vectors start at the **origin (0,0)**, and all of the words lie in the **upper-right part of the space**.

Because of this, many vectors end up pointing in roughly the **same direction**, even if the words are not actually very similar.

For example, consider the vectors for:

- **boy** → [1, 2]
- **child** → [5, 2]
- **adult** → [5, 7]

Even though **boy** is clearly closer in meaning to **child**, the vectors for **boy** and **adult** point in a more similar direction from the origin.

This means the **angle between "boy" and "adult" can actually be smaller** than the angle between **"boy" and "child"**, which would incorrectly suggest that **boy is more similar to adult**.

To fix this problem, we shift the entire space so that the **center of all the points becomes the origin**.

Recentering the Space

We compute the **mean (average)** of all the coordinates and subtract it from every point.

```
In [8]: # Create a new words_2d dictionary with zero-mean coordinates

# Compute the mean for each dimension
mean_x = sum(v[0] for v in words_2d.values()) / len(words_2d)
mean_y = sum(v[1] for v in words_2d.values()) / len(words_2d)

# Subtract the mean from each coordinate
words_2d_zero_mean = {
    word: [x - mean_x, y - mean_y]
    for word, (x, y) in words_2d.items()
}

# Show the new zero-mean coordinates, rounded to 2 decimal places
for word, coords in words_2d_zero_mean.items():
    print(f"{word}: [{coords[0]:.2f}, {coords[1]:.2f}]")

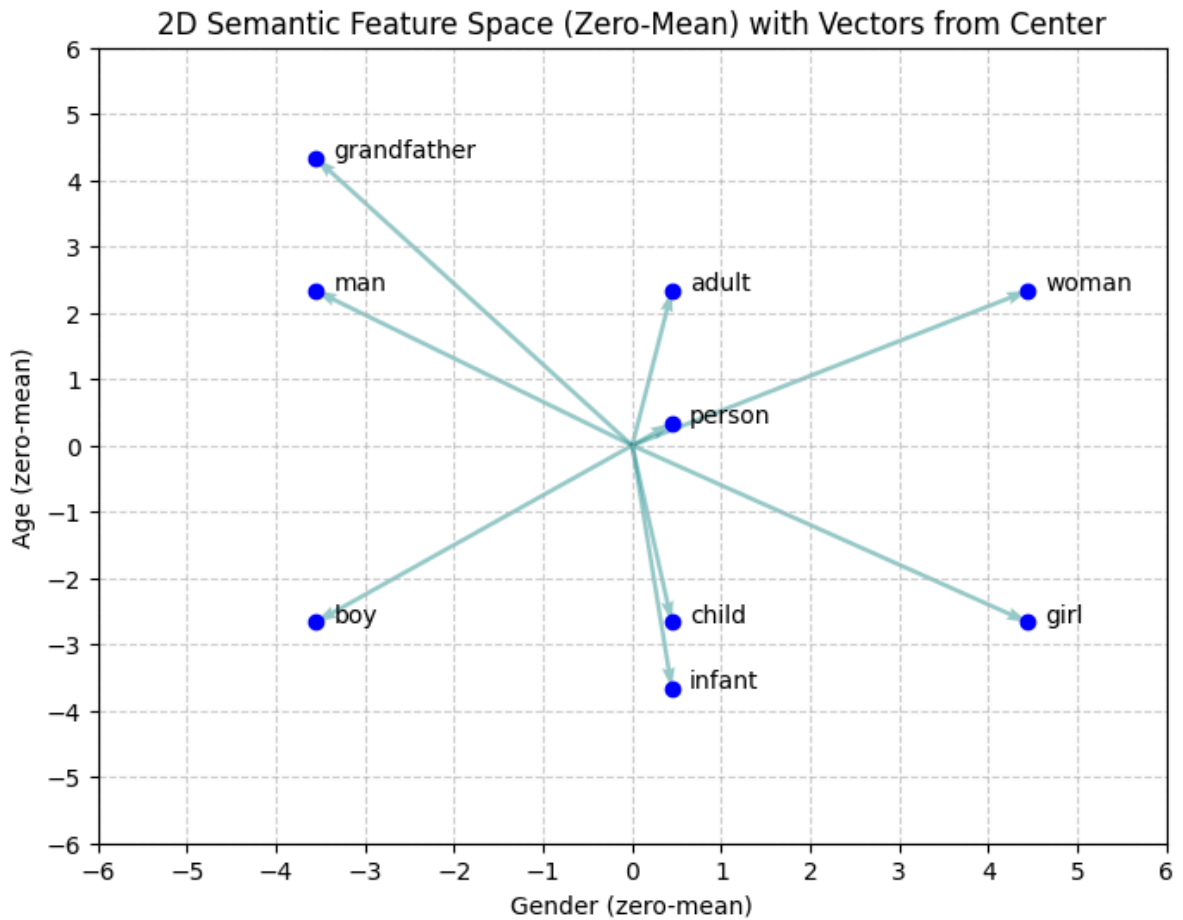
man: [-3.56, 2.33]
woman: [4.44, 2.33]
boy: [-3.56, -2.67]
girl: [4.44, -2.67]
person: [0.44, 0.33]
grandfather: [-3.56, 4.33]
adult: [0.44, 2.33]
child: [0.44, -2.67]
infant: [0.44, -3.67]
```

```
In [9]: # Plot words as points and vectors from the new origin (centered/zero-mean)
plt.figure(figsize=(8, 6))

for word, (x, y) in words_2d_zero_mean.items():
    # Vector arrow from origin (0,0) to the zero-mean word point
    plt.quiver(0, 0, x, y, angles='xy', scale_units='xy', scale=1,
               color='teal', alpha=0.4, width=0.004)
    # Point + label
    plt.scatter(x, y, color='blue')
    plt.text(x + 0.2, y, word, fontsize=10)

plt.xlim(-6, 6)
plt.ylim(-6, 6)
plt.xlabel("Gender (zero-mean)")
plt.ylabel("Age (zero-mean)")
plt.xticks(range(-6, 7, 1))
plt.yticks(range(-6, 7, 1))
```

```
plt.title("2D Semantic Feature Space (Zero-Mean) with Vectors from Center")
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



One More Adjustment: Normalizing the Vectors

After shifting the space so that the center is at the origin, the angles between vectors behave more sensibly. For example, the angle between **boy** and **child** is now smaller than the angle between **boy** and **adult**, which better reflects their meanings.

However, there is still one issue.

The dot product between two vectors does not depend only on the **angle between them**. It also depends on the **lengths of the vectors**.

Mathematically, the dot product can be written in two equivalent ways:

$$u \cdot v = x_1x_2 + y_1y_2 = \|u\| \|v\| \cos(\theta)$$

The first expression,

$$x_1x_2 + y_1y_2$$

comes directly from multiplying the coordinates of the vectors.

The second expression,

$$\|u\| \|v\| \cos(\theta)$$

comes from geometry. Here:

- $\|u\|$ and $\|v\|$ are the **lengths of the vectors**
- θ is the **angle between them**
- $\cos(\theta)$ measures how aligned the vectors are

These two formulas are mathematically **equivalent ways of computing the same quantity**. One uses coordinates, while the other describes the same result using geometry.

Because the dot product depends on vector **lengths**, two vectors that point in the same direction but have different lengths can produce different values.

To make similarity depend **only on direction**, we normalize each vector so that its length becomes **1**.

Given a vector

$$u = [x, y]$$

with length

$$\|u\| = \sqrt{x^2 + y^2}$$

we create a **unit vector**

$$u/\|u\| = [x/\|u\|, y/\|u\|]$$

After normalization, all vectors lie on a **circle of radius 1**, and the dot product becomes exactly the **cosine similarity** between the vectors.

```
In [10]: # Convert zero-mean points to unit vectors (all points lie on a circle of radius 1)
unit_vectors = {}
for word, (x, y) in words_2d_zero_mean.items():
    mag = math.sqrt(x**2 + y**2)
    if mag == 0:
        unit_vectors[word] = [0, 0]
    else:
        unit_vectors[word] = [x / mag, y / mag]

# Show the unit vectors, rounded to 2 decimal places
for word, coords in unit_vectors.items():
    print(f"{word}: [{coords[0]:.2f}, {coords[1]:.2f}]")
```

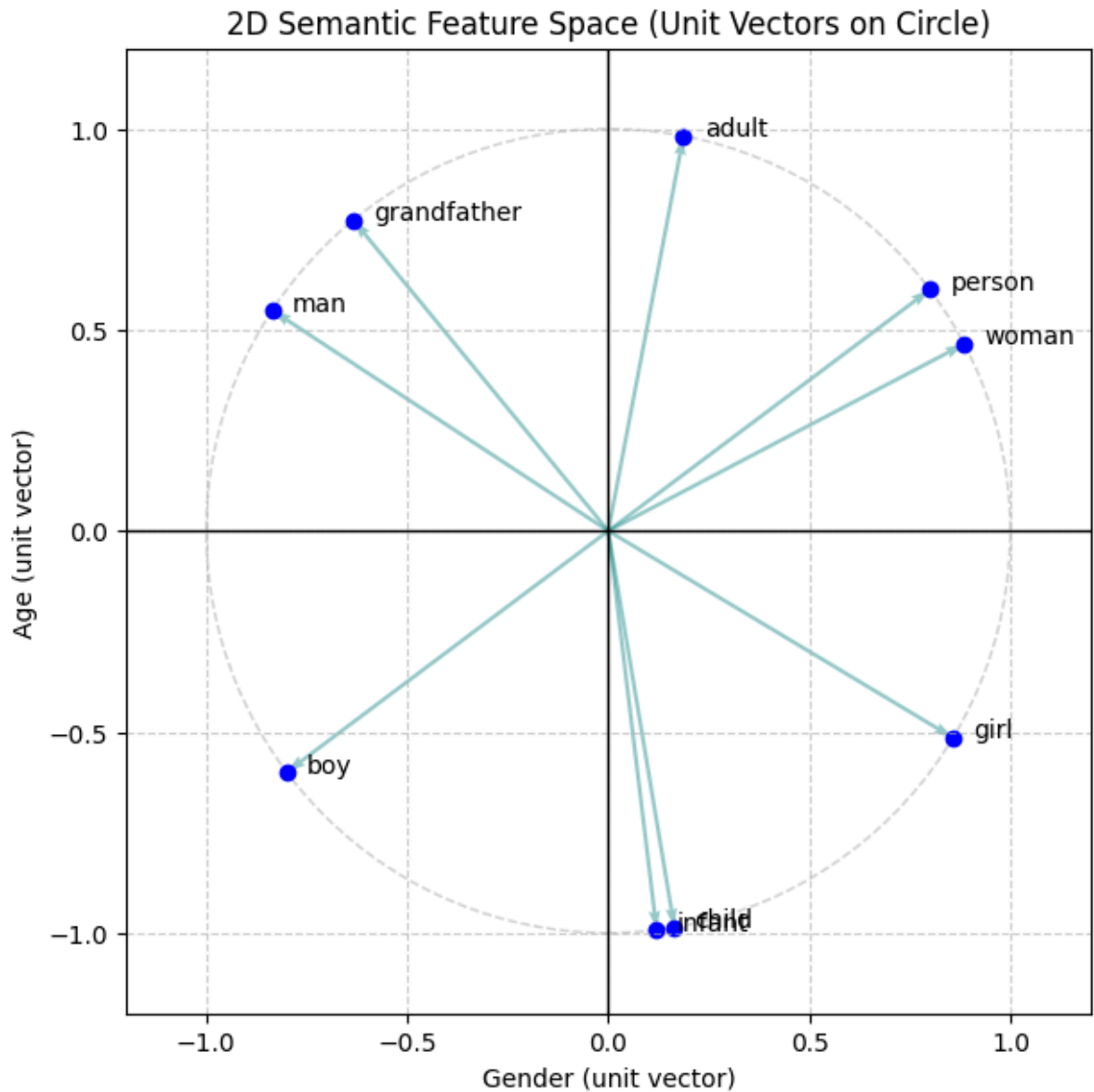
```
man: [-0.84, 0.55]
woman: [0.89, 0.46]
boy: [-0.80, -0.60]
girl: [0.86, -0.51]
person: [0.80, 0.60]
grandfather: [-0.63, 0.77]
adult: [0.19, 0.98]
child: [0.16, -0.99]
infant: [0.12, -0.99]
```

```
In [11]: # Plot unit vectors (zero-mean, length 1) for all words
plt.figure(figsize=(7, 7))

# Draw a circle of radius 1 for reference
circle = plt.Circle((0, 0), 1, color='lightgray', fill=False, linestyle='--')
plt.gca().add_patch(circle)

for word, (x, y) in unit_vectors.items():
    # Vector arrow from origin to the unit vector point
    plt.quiver(0, 0, x, y, angles='xy', scale_units='xy', scale=1,
               color='teal', alpha=0.4, width=0.004)
    # Point + label
    plt.scatter(x, y, color='blue')
    plt.text(x + 0.05, y, word, fontsize=10)

plt.xlim(-1.2, 1.2)
plt.ylim(-1.2, 1.2)
plt.xlabel("Gender (unit vector)")
plt.ylabel("Age (unit vector)")
plt.title("2D Semantic Feature Space (Unit Vectors on Circle)")
plt.axhline(0, color='black', lw=1)
plt.axvline(0, color='black', lw=1)
plt.grid(True, linestyle='--', alpha=0.6)
plt.gca().set_aspect('equal')
plt.show()
```



Adding Complexity: Royalty

Now let's consider the words **king**, **queen**, **prince**, and **princess**.

They share the same **gender** and **age** attributes as **man**, **woman**, **boy**, and **girl**, but they do **not** mean the same thing.

To distinguish them, we need a new semantic feature: **royalty**.

This means we now move from a **2D semantic space** to a **3D semantic space**.

Word	Gender (x)	Age (y)	Royalty (z)	Coordinates
man	1	7	1	[1, 7, 1]
woman	9	7	1	[9, 7, 1]
boy	1	2	1	[1, 2, 1]

Word	Gender (x)	Age (y)	Royalty (z)	Coordinates
girl	9	2	1	[9, 2, 1]
king	1	8	8	[1, 8, 8]
queen	9	7	8	[9, 7, 8]
prince	1	2	8	[1, 2, 8]
princess	9	2	8	[9, 2, 8]

```
In [12]: words_3d = {
    "man": [1, 7, 1],
    "woman": [9, 7, 1],
    "boy": [1, 2, 1],
    "girl": [9, 2, 1],
    "king": [1, 8, 8],
    "queen": [9, 7, 8],
    "prince": [1, 2, 8],
    "princess": [9, 2, 8]
}
```

```
In [13]: import plotly.graph_objects as go
import numpy as np

def plot_words_3d(words_3d,
                  arrows=None,
                  highlight_points=None,
                  colorscale="Viridis",
                  title="3D Semantic Feature Space"):
    """
    words_3d: dict of word -> [x, y, z]
    arrows: list of dicts with keys 'start', 'end', 'color', 'name', 'dash'
    highlight_points: list of dicts with keys 'word', 'color', 'size', 'symbol'
    """
    labels = list(words_3d.keys())
    xs = [coords[0] for coords in words_3d.values()]
    ys = [coords[1] for coords in words_3d.values()]
    zs = [coords[2] for coords in words_3d.values()]

    fig = go.Figure()

    # Scatter points (all words)
    fig.add_trace(go.Scatter3d(
        x=xs, y=ys, z=zs,
        mode="markers+text",
        text=labels,
        textposition="top center",
        marker=dict(size=6, color=zs, colorscale=colorscale, opacity=0.85),
        showlegend=False
    ))

    # Draw arrows if provided
    if arrows:
        for arrow in arrows:
            start = np.array(words_3d[arrow['start']])
```

```

end = np.array(words_3d[arrow['end']])
fig.add_trace(go.Scatter3d(
    x=[start[0], end[0]], y=[start[1], end[1]], z=[start[2], end[2]],
    mode="lines",
    line=dict(
        color=arrow.get('color', 'black'),
        width=arrow.get('width', 4),
        dash=arrow.get('dash', None)
    ),
    name=arrow.get('name', f"{arrow['start']}→{arrow['end']}")
))

# Highlight points if provided
if highlight_points:
    for pt in highlight_points:
        coords = pt['coords'] if 'coords' in pt else words_3d[pt['word']]
        fig.add_trace(go.Scatter3d(
            x=[coords[0]], y=[coords[1]], z=[coords[2]],
            mode="markers+text",
            marker=dict(
                size=pt.get('size', 8),
                color=pt.get('color', 'gold'),
                symbol=pt.get('symbol', 'x')
            ),
            text=[pt.get('label', pt.get('word', ''))],
            textposition="top center",
            name=pt.get('name', pt.get('word', '')),
            showlegend=pt.get('showlegend', False)
        ))

fig.update_layout(
    title=title,
    scene=dict(
        xaxis_title="Gender (x)",
        yaxis_title="Age (y)",
        zaxis_title="Royalty (z)",
        xaxis=dict(range=[0, 10]),
        yaxis=dict(range=[0, 10]),
        zaxis=dict(range=[0, 10]),
    ),
    width=900,
    height=700,
    legend=dict(itemsizing='constant')
)
fig.show()

# --- Usage Example 1: Just the points ---
plot_words_3d(words_3d, colorscale="Rainbow", title="Interactive 3D Semantic

```

Word Embeddings

Going from two semantic features to three allowed us to represent more words.

But are three features enough to represent words such as **cucumber**, **smiled**, or **honesty**? Probably not!

To represent the complexity of a large vocabulary, we would need **many more semantic features**.

Instead of designing these features by hand, we can let a machine learning algorithm learn them automatically from large amounts of text. These learned vector representations are called **word embeddings**.

A typical word embedding might use a space with **hundreds of dimensions**, so each word is represented by hundreds of numbers.

These learned representations support:

- similarity
- analogies
- search
- recommendation
- modern language models

How Are Word Embeddings Learned?

In early models such as *word2vec*, embeddings are learned using a simple neural network.

Each word in the vocabulary is first represented as a **one-hot vector**. A one-hot vector has a 1 in the position of the word and 0 everywhere else.

For example, if the vocabulary is:

[cat, dog, apple, car]

then the word **dog** might be represented as:

[0, 1, 0, 0]

A neural network then learns to **encode this one-hot vector into a dense vector**. This dense vector is the **word embedding**.

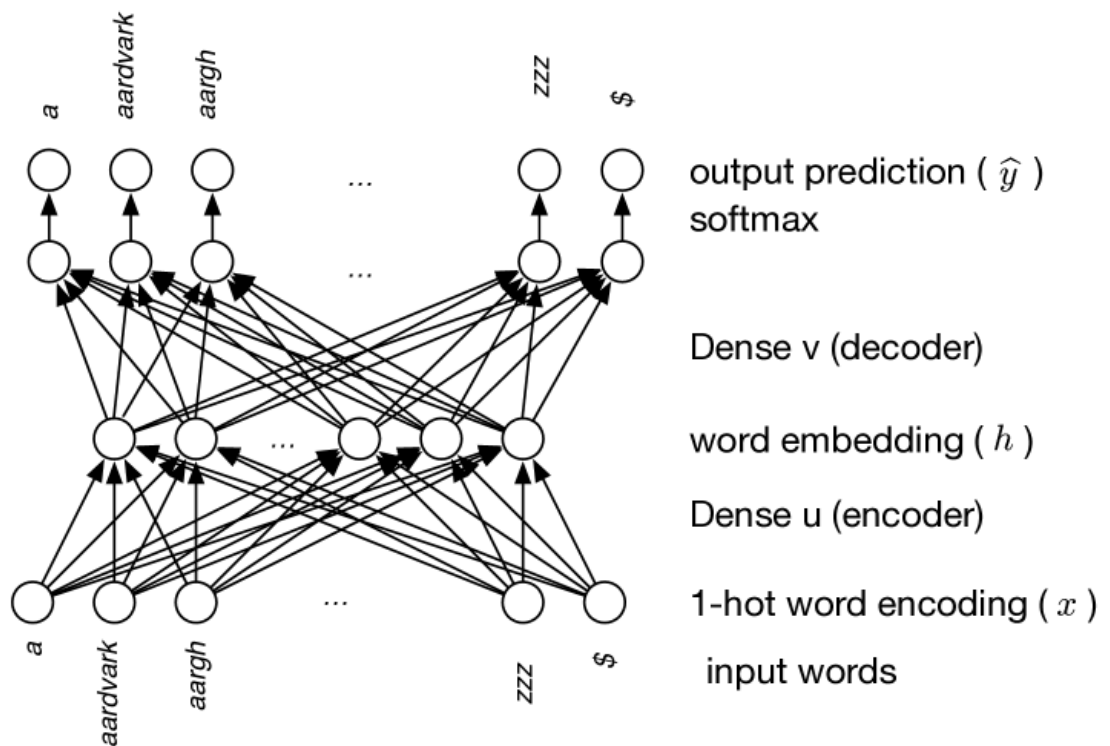


image source: <https://artint.info/3e/html/ArtInt3e.Ch8.S5.html>

The model learns these embeddings by predicting words that appear near each other in text.

Two common training methods are:

- *CBOW (Continuous Bag of Words)*: The model predicts a word from its surrounding context words.
- *Skip-gram*: The model predicts surrounding words from a single word.

By training on large text corpora, the model learns embeddings where *similar words end up close together in the vector space*.

Activity: Investigating Bias in Word Embeddings

Word embeddings are learned from large text corpora written by humans. As a result, they can also reflect **patterns and biases present in the data**.

In this activity, we will explore how bias can appear in embedding spaces.

Open the interactive tool: <https://lamiyowce.github.io/word2viz/>

Using the **gender analogy explorer**, search for the following words and observe the nearest neighbors:

- doctor
- nurse

- engineer
- teacher
- homemaker
- programmer

Q. What kinds of words appear near each profession?

YOUR ANSWER HERE

Q. Try exploring other words and analogies. Can you find any other types of bias in the data?

YOUR ANSWER HERE

Further Reading

If you are curious to learn more about the ideas behind the systems we explored in this lecture:

- Mikolov et al., *Efficient Estimation of Word Representations in Vector Space* - [Paper Link](#)
- Cer et al., *Universal Sentence Encoder* - [Paper Link](#)

(Semantris is based on this work)

- [GloVe](#) (Global Vectors for Word Representation) – Stanford NLP

Interactive Tools

- [Word Embedding Demo](#)
- [Word2Viz](#) : Interactive Embedding Explorer based on [GloVe](#)

Acknowledgment

Parts of this lecture are based on [Word Embedding Demo](#) developed at Carnegie Mellon University by Touretzky, D.